

PRACTICAL 1: Handling Missing Values

1. Aim / Objective

To detect missing values in a dataset and apply various imputation techniques including Mean, Median, Mode, SimpleImputer, and KNNImputer to handle them effectively.

2. Theory

► Simple Explanation

Imagine you collected survey data from 100 students. Some students skipped a few questions. Those blank answers are called "missing values." Machine learning models cannot work with blanks — they need numbers. So we either remove rows with blanks or fill them intelligently. Filling missing data is called "Imputation."

► Technical Explanation

Missing data in a dataset is represented as NaN (Not a Number) in Python/Pandas. There are three types:

- MCAR – Missing Completely at Random: No pattern in missing data
- MAR – Missing at Random: Missing values depend on observed data
- MNAR – Missing Not at Random: Missing values have a pattern/reason

► Why is it Important?

Missing values cause errors during model training, reduce accuracy, and may introduce bias if not handled properly. Most ML algorithms cannot process NaN values directly.

► Real-World Applications

- Healthcare: Patient vitals or test results often missing
- Finance: Credit data, transaction history with gaps
- E-commerce: Product ratings or customer reviews missing

3. Flowchart

```
START
↓
Load Dataset
↓
Detect Missing Values (isnull().sum())
↓
Decide Strategy: Remove OR Impute?
↓
If Remove → dropna() → END
↓
If Impute → Choose Method:
    |— Mean (Numerical, Normal dist.)
    |— Median (Numerical, Skewed dist.)
    |— Mode (Categorical)
    |— SimpleImputer (sklearn)
```

```
└─ KNNImputer (sklearn)
↓
Apply Imputation
↓
Verify: isnull().sum() == 0
↓
DONE - Clean Dataset Ready
END
```

4. Methods / Techniques

► 4.1 Mean Imputation

Definition: Replace missing values with the arithmetic mean of that column.

Formula: Mean = (Sum of all values) / (Count of non-null values)

Advantages: Simple, fast, works well for normally distributed data.

Disadvantages: Distorts variance; not suitable for skewed data.

Use Case: Age, salary in symmetric distributions.

► 4.2 Median Imputation

Definition: Replace missing values with the middle value of the sorted column.

Advantages: Robust to outliers; best for skewed distributions.

Disadvantages: Ignores feature correlations.

Use Case: Income, house prices (right-skewed data).

► 4.3 Mode Imputation

Definition: Replace missing values with the most frequently occurring value.

Advantages: Works for categorical and numerical data.

Disadvantages: May introduce bias if one class is dominant.

Use Case: Gender, city, product category columns.

► 4.4 SimpleImputer (scikit-learn)

A sklearn transformer that handles missing values using mean/median/mode/constant strategies. It integrates with ML Pipelines.

Parameters:

- strategy: "mean", "median", "most_frequent", "constant"
- fill_value: value to fill when strategy="constant"
- missing_values: default is np.nan

► 4.5 KNNImputer (scikit-learn)

Fills missing values using the average of K-Nearest Neighbors. More intelligent than mean/median — considers feature correlations.

Formula: Uses Euclidean distance to find K nearest rows with complete data, then averages their values.

Parameters:

- n_neighbors: Number of neighbors (default=5)
- weights: "uniform" or "distance"

- metric: "nan_euclidean" (handles NaN)

⚠ *KNNImputer is slower for large datasets but more accurate than simple strategies.*

5. Python Code

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer, KNNImputer

# — STEP 1: Load any CSV dataset —
df = pd.read_csv("your_dataset.csv")

# — STEP 2: Detect missing values —
print("Missing Values Count:")
print(df.isnull().sum())
print(f"Total Missing: {df.isnull().sum().sum()}")

# — STEP 3: Separate numerical and categorical columns —
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = df.select_dtypes(include=["object", "category"]).columns.tolist()

# — STEP 4: Mean Imputation —
df_mean = df.copy()
df_mean[num_cols] = df_mean[num_cols].fillna(df_mean[num_cols].mean())

# — STEP 5: Median Imputation —
df_median = df.copy()
df_median[num_cols] = df_median[num_cols].fillna(df_median[num_cols].median())

# — STEP 6: Mode Imputation (for categorical) —
df_mode = df.copy()
df_mode[cat_cols] = df_mode[cat_cols].fillna(df_mode[cat_cols].mode().iloc[0])

# — STEP 7: SimpleImputer —
si = SimpleImputer(strategy="mean") # Change strategy as needed
df_simple = df.copy()
df_simple[num_cols] = si.fit_transform(df[num_cols])

# — STEP 8: KNNImputer —
knn = KNNImputer(n_neighbors=5)
df_knn = df.copy()
df_knn[num_cols] = knn.fit_transform(df[num_cols])

# — STEP 9: Verify no missing values remain —
print("After Imputation:", df_knn.isnull().sum().sum(), "missing values")
```

6. Code Explanation

Step-by-Step Breakdown:

- `df.isnull().sum()` → Counts NaN in each column; `.sum().sum()` gives total missing
- `select_dtypes()` → Automatically identifies numerical vs categorical columns
- `fillna(mean())` → Replaces NaN with column mean in one line

- `SimpleImputer.fit_transform()` → Learns mean from training data then applies it
- `KNNImputer.fit_transform()` → Finds 5 nearest complete rows and averages their values

💡 *Always split train/test BEFORE imputation to prevent data leakage!*

7. Output Explanation

After running the code, you will see:

- A table showing how many values are missing per column (before)
- After imputation: 0 missing values confirmed
- Mean-imputed data: NaN replaced by average (e.g., Age NaN → 28.5)
- KNN-imputed data: NaN replaced by neighbor-average (more contextual)

8. Technique Comparison Table

Method	Data Type	Handles Outliers	Complexity	Accuracy	Speed
Mean	Numerical	No	Low	Moderate	Fast
Median	Numerical	Yes	Low	Good	Fast
Mode	Categorical	Moderate	Low	Moderate	Fast
SimpleImputer	Both	Depends	Low	Moderate	Fast
KNNImputer	Numerical	Yes	High	High	Slow

9. Advantages & Limitations

Advantages:

- Preserves dataset size (no row deletion)
- KNNImputer captures feature correlations
- SimpleImputer integrates with sklearn Pipelines

Limitations:

- Mean/Median reduces variance → less informative data
- KNNImputer is $O(n^2)$ in complexity — slow for large data
- Wrong strategy can introduce bias

10. Applications

- Medical records: Fill missing lab test results
- Weather data: Impute missing sensor readings
- Customer churn: Handle missing usage metrics

11. Viva Questions & Answers

Q1. What is imputation?

A: The process of replacing missing values in a dataset with estimated values based on available data.

Q2. When should you use Median instead of Mean?

A: When data is skewed or has outliers. Median is not affected by extreme values.

Q3. What is the difference between SimpleImputer and KNNImputer?

A: SimpleImputer uses global statistics (mean/median/mode). KNNImputer uses K nearest similar rows to estimate missing values — it is more accurate but slower.

Q4. What is data leakage in imputation?

A: Fitting the imputer on the entire dataset before train-test split causes leakage — the model learns test set statistics. Always fit on training data only.

Q5. What are MCAR, MAR, and MNAR?

A: MCAR = missing randomly with no pattern. MAR = missing depends on observed data. MNAR = missingness has a systematic cause.

Q6. Which imputation works for categorical data?

A: Mode imputation or SimpleImputer with strategy="most_frequent".

Q7. What is the simplest way to handle missing values?

A: df.dropna() — removes all rows with any missing value. But this reduces dataset size.

12. Conclusion

Missing value handling is the first and most critical step in data preprocessing. Mean and Median are quick fixes for numerical data; Mode handles categorical data. KNNImputer provides the most accurate estimates by considering feature relationships. Always choose the technique based on data distribution, percentage of missing data, and data type.

 **Best Practice:** Use Median for skewed data, KNNImputer when accuracy matters, and always impute AFTER train-test split.

PRACTICAL 2: Data Normalization

1. Aim / Objective

To apply Min-Max Scaling and Standard Scaling (Z-score Normalization) on datasets and compare their effects on Machine Learning model performance.

2. Theory

► Simple Explanation

Imagine measuring student height in cm (150–180) and weight in kg (50–90). Height values are always larger than weight. A distance-based model like KNN will be biased toward height. Normalization brings all features to the same scale so no single feature dominates.

► Technical Explanation

Feature values with different ranges cause gradient descent to converge slowly and bias distance-based algorithms. Normalization (or Standardization) rescales features to a common range.

► Real-World Applications

- Image processing: Pixel values normalized 0–1
- Neural networks: Inputs normalized for stable training
- KNN/SVM: Distance-based — require equal scale features

3. Flowchart

```
START
↓
Load Dataset with numerical features
↓
Split into Train / Test sets
↓
Choose Normalization Method:
  ├── Min-Max Scaling (output range 0 to 1)
  └── Standard Scaling (output: mean=0, std=1)
↓
Fit scaler on TRAIN set only
↓
Transform TRAIN and TEST sets
↓
Train ML model on scaled data
↓
Evaluate: Accuracy / RMSE
↓
Compare with Un-scaled model
END
```

4. Methods / Techniques

► 4.1 Min-Max Scaling (MinMaxScaler)

Rescales data to [0, 1] range.

Formula: $X_{\text{scaled}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$

Advantages: Bounded output [0,1]; good for image data and neural networks.

Disadvantages: Sensitive to outliers (extreme values stretch the range).

Use Case: CNN inputs, data without extreme outliers.

► 4.2 Standard Scaling (StandardScaler)

Centers data to mean=0 and scales to unit variance (std=1).

Formula: $X_{\text{scaled}} = (X - \text{mean}) / \text{std_deviation}$

Advantages: Not affected by scale; works well for algorithms assuming normal distribution (SVM, Logistic Regression, PCA).

Disadvantages: Output is unbounded; not always [0,1].

Use Case: SVM, PCA, Logistic Regression, Linear models.

5. Python Code

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# — Load and prepare data —
df = pd.read_csv("your_dataset.csv")
X = df.drop("target", axis=1).select_dtypes(include=[np.number])
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# — No Scaling (baseline) —
model = KNeighborsClassifier()
model.fit(X_train, y_train)
acc_raw = accuracy_score(y_test, model.predict(X_test))
print(f"Accuracy (No Scaling): {acc_raw:.4f}")

# — Min-Max Scaling —
scaler_mm = MinMaxScaler()
X_train_mm = scaler_mm.fit_transform(X_train) # fit on train only
X_test_mm = scaler_mm.transform(X_test)      # only transform test
model.fit(X_train_mm, y_train)
acc_mm = accuracy_score(y_test, model.predict(X_test_mm))
print(f"Accuracy (Min-Max): {acc_mm:.4f}")

# — Standard Scaling —
scaler_ss = StandardScaler()
X_train_ss = scaler_ss.fit_transform(X_train)
X_test_ss = scaler_ss.transform(X_test)
model.fit(X_train_ss, y_train)
acc_ss = accuracy_score(y_test, model.predict(X_test_ss))
```

```
|print(f"Accuracy (Standard): {acc_ss:.4f}")
```

6. Code Explanation

- `fit_transform(X_train)` → Learns statistics FROM training data and applies scaling
- `transform(X_test)` → Applies SAME statistics to test data (no re-learning)
- Never fit on test set → avoids data leakage
- `KNeighborsClassifier` used as test model because it is highly sensitive to scale

7. Output Explanation

Expected output example:

- Accuracy (No Scaling): 0.7200
- Accuracy (Min-Max): 0.8900
- Accuracy (Standard): 0.8750

Scaling significantly improves KNN performance. The improvement depends on feature range differences.

8. Comparison Table

Property	No Scaling	Min-Max	Standard
Output Range	Original	[0, 1]	~[-3, 3]
Handles Outliers	—	No	Somewhat
Best For	Tree models	Neural Nets, KNN	SVM, PCA, LR
Preserves Distribution	Yes	Yes	Yes

9. Advantages & Limitations

Min-Max:

- Good for bounded data
- Sensitive to outliers — one extreme value ruins scaling

Standard:

- Better for outlier-prone data
- Output range not bounded — may not suit all algorithms

10. Applications

- Min-Max: Image pixel normalization, ANN inputs
- Standard: SVM, PCA, Logistic Regression, Linear models

11. Viva Questions & Answers

Q1. What is the difference between normalization and standardization?

A: Normalization scales to [0,1] range (Min-Max). Standardization scales to mean=0, std=1 (Z-score). Both remove scale differences but differently.

Q2. Which algorithms DO NOT need scaling?

A: Decision Trees, Random Forest, Gradient Boosting, XGBoost — tree-based models are not affected by feature scale.

Q3. Why should we never fit the scaler on test data?

A: Fitting on test data introduces data leakage — the model gets information from test set statistics, making evaluation unreliable.

Q4. What happens without normalization in KNN?

A: Features with larger ranges dominate the distance calculation, causing the model to ignore smaller-ranged but equally important features.

Q5. What does StandardScaler do to skewed data?

A: It centers and scales but does NOT fix skewness. For skewed data, use PowerTransformer or log transform first.

12. Conclusion

Normalization is critical for distance-based and gradient-based ML algorithms. Min-Max Scaling is ideal for bounded, clean data. Standard Scaling is preferred for data with outliers and algorithms assuming Gaussian distribution. Always apply scalers only on training data and transform test data separately.

 *Rule of Thumb: Use StandardScaler for SVM/PCA. Use MinMaxScaler for Neural Networks. Skip for tree models.*

PRACTICAL 3: Feature Scaling

1. Aim / Objective

To apply Log Scaling, Power Transform, and Robust Scaling on skewed datasets and compare their effects on data distribution and model performance.

2. Theory

► Simple Explanation

Some data is "lopsided" — like income data where most people earn ₹20,000–50,000 but a few earn ₹10 Crore. This skew confuses ML models. Special scaling methods like log transform or power transform fix this by pulling extreme values closer.

► Technical Explanation

Skewness in data violates the normality assumptions of linear models and reduces performance of many algorithms. Feature scaling methods like log transform and power transforms normalize skewed distributions.

3. Flowchart

```
START
↓
Load Dataset
↓
Check Skewness: df.skew()
↓
Skewness > 0.5 or < -0.5? → Yes → Apply Transform
↓
Choose Transform:
├── Log Transform      → np.log1p(X)  [Right skew]
├── Power Transform    → PowerTransformer() [Any skew]
└── Robust Scaling     → RobustScaler() [With Outliers]
↓
Fit on Train, Transform Train & Test
↓
Visualize: Histogram before vs. after
↓
Train model → Compare metrics
END
```

4. Methods / Techniques

► 4.1 Log Scaling

Applies natural log to reduce right-skewed (positive skew) data.

Formula: $X_{\text{new}} = \log(1 + X)$ [np.log1p handles zeros]

Advantages: Simple, effective for right-skewed data like price, salary, population.

Disadvantages: Cannot handle negative values. Only works for right skew.


```

        ("Robust", X_train_rs, X_test_rs)]:
    lr = LinearRegression()
    lr.fit(Xtr, y_train)
    rmse = mean_squared_error(y_test, lr.predict(Xte), squared=False)
    print(f"{name} RMSE: {rmse:.4f}")

```

6. Code Explanation

- `np.log1p()` → $\log(1+X)$, handles zero values safely
- `.clip(lower=0)` → ensures no negative values before log
- `PowerTransformer` with `yeo-johnson` → works for negatives; finds optimal lambda
- `RobustScaler` → uses median & IQR so outliers don't affect scaling
- `RMSE` (Root Mean Squared Error) → error metric; lower is better

7. Output Explanation

Expected output:

- Raw RMSE: 45.23 (high error due to skew)
- Log RMSE: 32.10 (improved)
- Power RMSE: 29.45 (best for symmetric result)
- Robust RMSE: 31.88 (good with outliers present)

`PowerTransform` usually gives best performance as it finds optimal transformation per column.

8. Comparison Table

Property	Log Scale	Power Transform	Robust Scale	Min-Max
Handles Outliers	No	Moderate	Yes	No
Handles Negatives	No	Yeo-Johnson: Yes	Yes	Yes
Fixes Skewness	Right Skew	Both Directions	No	No
Complexity	Low	Medium	Low	Low

9. Advantages & Limitations

- **Log:** Simple and interpretable but fails for negative/zero data
- **Power Transform:** Automatic and flexible; adds model complexity
- **Robust Scaler:** Best for outlier-heavy data; does not remove outliers

10. Applications

- **Log:** Financial data (stock prices, revenue), population statistics
- **Power:** Medical/biological measurements, sensor data
- **Robust:** Salary, real estate prices, crime statistics

11. Viva Questions & Answers

Q1. What is skewness?

A: Skewness measures asymmetry in data distribution. Positive skew = long right tail. Negative skew = long left tail. Ideal ML data has skewness near 0.

Q2. Why cant log transform handle negative values?

A: $\log(\text{negative})$ is undefined mathematically. Use Yeo-Johnson (PowerTransformer) instead, which handles all values.

Q3. What is IQR?

A: Interquartile Range = $Q3 - Q1$ (75th percentile minus 25th percentile). It represents the middle 50% of data and is robust to outliers.

Q4. Which transform to use for negatively skewed data?

A: PowerTransformer with yeo-johnson method. Log transform only works for right (positive) skew.

Q5. Does RobustScaler remove outliers?

A: No. It scales data to make the model LESS SENSITIVE to outliers but does not remove them from the dataset.

12. Conclusion

Feature scaling is essential for skewed datasets. Log Transform is simple and effective for right-skewed positive data. PowerTransformer is the most flexible and handles both types of skew automatically. RobustScaler is the best choice when outliers are present. Always check skewness using `df.skew()` before choosing a method.

 *Tip: Skewness > 0.5 → apply transformation. Use `df.hist()` to visualize before & after.*

PRACTICAL 4: Encoding Categorical Variables

1. Aim / Objective

To convert categorical text data into numerical format using One-Hot Encoding, Label Encoding, and Ordinal Encoding; and to understand when to apply each technique.

2. Theory

► Simple Explanation

ML models work only with numbers. But many real datasets have text — like "Male/Female", "Red/Blue/Green", or "Low/Medium/High". We must convert these to numbers. But simply assigning 1,2,3 to categories may imply an order that doesn't exist. Different encoding strategies handle this differently.

► Technical Explanation

Categorical variables can be nominal (no order: Color, City) or ordinal (with order: Low < Medium < High). The encoding method must respect or ignore this order based on the data type.

3. Flowchart

```
START
↓
Identify Categorical Columns
↓
Check: Does the category have ORDER?
↓
YES (Ordinal) → Use OrdinalEncoder
↓
NO (Nominal) → Check number of unique values
↓
Low Cardinality (< 10) → Use One-Hot Encoding (pd.get_dummies)
↓
High Cardinality (≥ 10) → Use Label Encoding or Target Encoding
↓
Apply chosen encoding
↓
Verify: df.dtypes shows int/float for all columns
END
```

4. Methods / Techniques

► 4.1 One-Hot Encoding

Creates a binary column for each unique category value.

Example: Color [Red, Blue, Green] → Red:[1,0,0] Blue:[0,1,0] Green:[0,0,1]

Advantages: No false ordinal relationship; ML-friendly.

Disadvantages: High cardinality (many unique values) → too many columns (curse of dimensionality).

Use Case: Nominal variables with few unique values (< 10–15 categories).

► 4.2 Label Encoding

Assigns integer values to each category alphabetically.

Example: Color → Red=2, Blue=0, Green=1

Advantages: Simple, no extra columns, works well with tree models.

Disadvantages: Implies a numerical order (Blue < Green < Red) that may mislead linear models.

Use Case: Tree-based models (Decision Tree, Random Forest) where order doesn't matter.

► 4.3 Ordinal Encoding

Encodes categories with a predefined meaningful order.

Example: Education [High School, Bachelor, Master, PhD] → [0, 1, 2, 3]

Advantages: Preserves true ordinal relationships.

Disadvantages: User must manually define the order.

Use Case: Truly ordered categories like ratings, education level, size (S/M/L/XL).

5. Python Code

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder

df = pd.read_csv("your_dataset.csv")
cat_cols = df.select_dtypes(include=["object", "category"]).columns.tolist()
print("Categorical columns:", cat_cols)

# — One-Hot Encoding —
df_ohe = pd.get_dummies(df, columns=cat_cols, drop_first=True)
print("Shape after OHE:", df_ohe.shape)

# — Label Encoding —
df_le = df.copy()
le = LabelEncoder()
for col in cat_cols:
    df_le[col] = le.fit_transform(df_le[col].astype(str))

# — Ordinal Encoding (define order manually) —
# Example for "size" column with order: Small < Medium < Large
ordinal_col = "size" # Change to your ordinal column name
categories_order = ["Small", "Medium", "Large"]
if ordinal_col in df.columns:
    oe = OrdinalEncoder(categories=categories_order)
    df_ord = df.copy()
    df_ord[[ordinal_col]] = oe.fit_transform(df_ord[[ordinal_col]])

print("Sample OHE output:")
print(df_ohe.head(3))
```

6. Code Explanation

- `pd.get_dummies()` → automatic one-hot encoding; `drop_first=True` removes one column to avoid multicollinearity
- `LabelEncoder` → encodes one column at a time alphabetically
- `OrdinalEncoder` → takes `categories=[list_of_ordered_values]` to define exact order
- `astype(str)` → converts NaN to string to avoid encoder errors

7. Output Explanation

One-Hot: Color (3 unique) → 2 new columns (`drop_first` removes 1 to prevent dummy variable trap)

Label: Red→2, Blue→0, Green→1 (alphabetical order)

Ordinal: Small→0, Medium→1, Large→2 (as defined by user)

8. Comparison Table

Property	One-Hot	Label	Ordinal	Best For
Creates New Cols	Yes	No	No	—
Implies Order	No	Yes (false)	Yes (true)	—
High Cardinality	Bad	Good	Medium	—
Tree Models	Works	Works	Works	Label/Ordinal
Linear Models	Best	Risky	If truly ordinal	OHE

11. Viva Questions & Answers

Q1. What is the dummy variable trap?

A: When using OHE with N categories, you need only N-1 columns. Having N columns creates perfect multicollinearity. Use `drop_first=True` to avoid it.

Q2. When is Label Encoding safe?

A: Only with tree-based models (Decision Trees, RF, XGBoost) that don't assume ordinal relationships. Never use Label Encoding with linear models.

Q3. What is cardinality?

A: The number of unique values in a categorical column. High cardinality = many unique values. Cities, Names, Product IDs have high cardinality.

Q4. What encoding to use for ZIP codes or IDs?

A: Neither OHE (too many columns) nor Label Encoding (false order). Use Target Encoding or embedding layers.

Q5. What is Target Encoding?

A: Replacing category with the mean of the target variable for that category. Effective for high-cardinality features.

12. Conclusion

Encoding categorical data is essential before applying any ML algorithm. One-Hot Encoding is ideal for nominal variables with few categories. Label Encoding is quick but risky for linear models. Ordinal Encoding should only be used when genuine order exists. Always match the encoding method to the algorithm and data type.

✓ Golden Rule: Nominal + Linear Model → One-Hot | Ordinal → OrdinalEncoder | Any + Tree Model → LabelEncoder is fine

PRACTICAL 5: Feature Extraction

1. Aim / Objective

To extract meaningful numerical features from text data using TF-IDF, understand Word Embeddings (Word2Vec/GloVe/FastText), and explain Image Feature Extraction using CNNs.

2. Theory

► Simple Explanation

Raw text and images cannot be directly fed to ML models. We must convert them into numerical vectors (called "feature vectors" or "embeddings"). TF-IDF converts text to numbers based on word frequency. Word2Vec learns semantic relationships. CNNs extract visual patterns from images.

3. Flowchart

```
START
↓
Raw Data: Text or Image
↓
TEXT PATH:
  Step 1: Preprocess (lowercase, remove stopwords, tokenize)
  Step 2: Choose Method:
    ├── TF-IDF → Sparse vector (frequency-based)
    └── Word Embedding → Dense vector (semantic)
↓
IMAGE PATH:
  Step 1: Resize image to fixed size (e.g. 224x224)
  Step 2: Pass through pretrained CNN
  Step 3: Extract features from penultimate layer
↓
Feature Vector → Feed to ML model
END
```

4. Methods / Techniques

► 4.1 TF-IDF (Term Frequency - Inverse Document Frequency)

Converts text documents to numerical vectors based on how important each word is in a document relative to the entire corpus.

TF(t,d) = Count of term t in document d / Total terms in d

IDF(t) = log(Total documents / Documents containing t)

TF-IDF = TF × IDF

High TF-IDF → word is important in THIS document but rare overall → meaningful feature.

Advantages: Simple, fast, interpretable, no training needed.

Disadvantages: Ignores word order and semantics. Sparse high-dimensional vectors.

Use Case: Document classification, spam detection, search engines.

► 4.2 Word Embeddings

Dense vector representations that capture semantic meaning. Similar words have similar vectors.

- Word2Vec: Predicts surrounding words (Skip-gram) or predicts a word from context (CBOW). Trained on large text corpora. Each word → 100–300 dimensional vector.
- GloVe: Uses global co-occurrence statistics. Word relationships encoded as vector arithmetic: King - Man + Woman ≈ Queen
- FastText: Like Word2Vec but processes subword (character n-grams). Handles typos and rare words better.

Advantage over TF-IDF: Captures semantic relationships ("good" ≈ "great").

► 4.3 CNN Image Feature Extraction

Pretrained CNNs (VGG16, ResNet, MobileNet) learned features from millions of images. We remove the final classification layer and use the output of the last dense layer as a feature vector.

Process: Raw Image → Resize → Normalize → CNN → Feature Vector (e.g., 2048-dim) → Your Classifier

This is called Transfer Learning — reusing learned image features for new tasks.

5. Python Code

Part A: TF-IDF

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Example: Text classification
df = pd.read_csv("text_dataset.csv") # needs "text" and "label" columns
X, y = df["text"], df["label"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# TF-IDF: Convert text to numerical features
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2), stop_words="english")
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

# Train and evaluate
lr = LogisticRegression(max_iter=500)
lr.fit(X_train_tfidf, y_train)
print(f"TF-IDF Accuracy: {accuracy_score(y_test, lr.predict(X_test_tfidf)):.4f}")
```

Part B: Word2Vec (Averaging embeddings)

```
# pip install gensim
from gensim.models import Word2Vec
import numpy as np

sentences = [text.lower().split() for text in df["text"]]
w2v_model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, epochs=10)

def get_doc_vector(words, model):
    vectors = [model.wv[w] for w in words if w in model.wv]
```

```

        return np.mean(vectors, axis=0) if vectors else np.zeros(100)

X_w2v = np.array([get_doc_vector(text.lower().split(), w2v_model) for text in
df["text"]])
print("Word2Vec feature matrix shape:", X_w2v.shape)

```

Part C: CNN Image Features

```

# pip install tensorflow
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.models import Model
import numpy as np

# Load pretrained CNN without top (classification) layer
base_model = MobileNetV2(weights="imagenet", include_top=False, pooling="avg")

def extract_image_features(img_path):
    img = load_img(img_path, target_size=(224, 224))
    arr = preprocess_input(img_to_array(img))
    return base_model.predict(np.expand_dims(arr, 0))[0]

# Usage: feature_vector = extract_image_features("image.jpg")
# feature_vector.shape → (1280,) ← 1280-dim feature vector

```

6. Code Explanation

- TfidfVectorizer: max_features=5000 limits to 5000 most important words; ngram_range=(1,2) captures single words and pairs
- Word2Vec: vector_size=100 → 100-dim embeddings; window=5 → considers 5 surrounding words
- get_doc_vector() → averages word vectors to get one vector per document
- MobileNetV2 with include_top=False → removes final softmax classifier → gives feature vector
- pooling="avg" → global average pooling reduces spatial dimensions to single vector

7. Output Explanation

- TF-IDF: Each document becomes a sparse vector (5000 dims). Accuracy depends on task.
- Word2Vec: Each document becomes a dense 100-dim vector. Captures meaning better.
- CNN: Each image becomes a 1280-dim vector ready for any classifier.

8. Comparison Table

Property	TF-IDF	Word2Vec	GloVe/FastText	CNN Features
Data Type	Text	Text	Text	Image
Semantic Aware	No	Yes	Yes	Yes
Vector Type	Sparse	Dense	Dense	Dense
Training Needed	No	Yes	Pretrained	Pretrained

11. Viva Questions & Answers

Q1. What does TF-IDF stand for?

A: Term Frequency - Inverse Document Frequency. It measures how important a word is to a document relative to the entire collection (corpus).

Q2. Why does TF-IDF penalize common words?

A: Words like "the", "is", "are" appear in all documents (high IDF denominator), so their TF-IDF score becomes very low. Rare but meaningful words get high scores.

Q3. What is an embedding?

A: A dense vector representation of data (word, image, user) in a continuous vector space where similar items are closer together.

Q4. What is transfer learning?

A: Using a model trained on a large dataset (like ImageNet) as a feature extractor for a different but related task. Saves time and requires less data.

Q5. How does Word2Vec capture meaning?

A: By training a neural network to predict surrounding words. Words appearing in similar contexts get similar vectors. Example: "King" and "Queen" are close in vector space.

12. Conclusion

Feature extraction converts raw unstructured data (text, images) into numerical vectors. TF-IDF is fast and interpretable for text but lacks semantic understanding. Word Embeddings capture meaning and relationships. CNN-based extraction transfers powerful visual features learned from millions of images to new tasks. Choosing the right extraction method determines model performance significantly.

PRACTICAL 6: Feature Selection

1. Aim / Objective

To identify the most important features using Correlation Analysis, Mutual Information, Recursive Feature Elimination (RFE), and Feature Importance methods.

2. Theory

► Simple Explanation

Not all columns in a dataset are useful. Some are redundant, some are irrelevant, and some may even hurt model performance (curse of dimensionality). Feature selection finds the BEST subset of features that gives the highest accuracy with fewer columns.

► Why is Feature Selection Important?

- Reduces overfitting
- Speeds up training
- Improves accuracy by removing noise
- Makes the model more interpretable

3. Flowchart

```
START
↓
Load Dataset (many features)
↓
Apply Feature Selection Method:
  ├── Filter Methods (independent of model):
  │   ├── Correlation Analysis
  │   └── Mutual Information
  ├── Wrapper Methods (uses model):
  │   └── Recursive Feature Elimination (RFE)
  └── Embedded Methods (built into model):
      └── Feature Importance (RF, XGBoost)
↓
Select Top-K features
↓
Train model on selected features
↓
Compare accuracy: All features vs Selected features
END
```

4. Methods / Techniques

► 4.1 Correlation Analysis

Measures linear relationship between features and target. Pearson correlation gives value between -1 and +1.

Formula: $r = \frac{\sum[(X_i - \bar{X})(Y_i - \bar{Y})]}{[n \times \sigma_X \times \sigma_Y]}$

$|r|$ close to 1 → Strong relationship; close to 0 → Weak or no relationship.

Also used to find correlated features → remove one of highly correlated pairs ($|r| > 0.9$) to reduce redundancy.

► 4.2 Mutual Information

Measures how much information one feature provides about the target. Works for both linear and non-linear relationships.

Formula: $MI(X;Y) = \sum P(x,y) \times \log[P(x,y) / (P(x) \times P(y))]$

MI = 0 → X and Y are independent. Higher MI → more information shared.

► 4.3 Recursive Feature Elimination (RFE)

Trains model on all features → removes least important feature → repeats until desired number of features reached. A wrapper method — uses a model internally.

Parameters: estimator (any sklearn model), n_features_to_select (number to keep), step (features to remove per iteration).

► 4.4 Feature Importance

Tree-based models (Random Forest, XGBoost) compute feature importance based on how much each feature reduces impurity across all trees. Built-in to model — no extra step needed.

5. Python Code

```
import pandas as pd
import numpy as np
from sklearn.feature_selection import mutual_info_classif, RFE
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.read_csv("your_dataset.csv")
X = df.drop("target", axis=1).select_dtypes(include=[np.number])
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# — 1. Correlation Analysis —
corr = X_train.corrwith(pd.Series(y_train,
name="target")).abs().sort_values(ascending=False)
top_corr_features = corr[corr > 0.1].index.tolist()
print("Top correlated features:", top_corr_features)

# Remove highly correlated feature pairs
corr_matrix = X_train.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
drop_cols = [col for col in upper.columns if any(upper[col] > 0.95)]
X_low_corr = X_train.drop(columns=drop_cols)

# — 2. Mutual Information —
mi_scores = mutual_info_classif(X_train, y_train, random_state=42)
mi_df = pd.Series(mi_scores, index=X_train.columns).sort_values(ascending=False)
```

```

top_mi = mi_df[mi_df > 0.01].index.tolist()
print("Top MI features:", top_mi)

# — 3. RFE —
lr = LogisticRegression(max_iter=500)
rfe = RFE(estimator=lr, n_features_to_select=min(10, X_train.shape[1]))
rfe.fit(X_train, y_train)
rfe_features = X_train.columns[rfe.support_].tolist()
print("RFE selected features:", rfe_features)

# — 4. Feature Importance (Random Forest) —
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
fi = pd.Series(rf.feature_importances_,
index=X_train.columns).sort_values(ascending=False)
top-fi = fi[fi > 0.01].index.tolist()
print("Top RF features:", top-fi)

# — Compare accuracy with all vs selected features —
for name, features in [("All", X_train.columns.tolist()), ("RFE", rfe_features), ("RF
Importance", top-fi)]:
    rf.fit(X_train[features], y_train)
    acc = accuracy_score(y_test[features] if False else y_test,
rf.predict(X_test[features]))
    print(f"Accuracy ({name} features): {acc:.4f}")

```

6. Code Explanation

- `corrwith()` → computes correlation of each feature with the target column
- `np.triu()` → upper triangle of correlation matrix → avoids counting pairs twice
- `mutual_info_classif()` → works for classification tasks; use `mutual_info_regression` for regression
- `RFE(n_features_to_select=10)` → keeps top 10 features as selected by model
- `rf.feature_importances_` → Gini importance: how much each feature reduces impurity

8. Comparison Table

Method	Type	Handles Non-linear	Speed	Best For
Correlation	Filter	No	Fast	Linear relationships
Mutual Info	Filter	Yes	Fast	Any relationship
RFE	Wrapper	Depends	Slow	Linear models
Feature Importance	Embedded	Yes	Fast	Tree models

11. Viva Questions & Answers

Q1. What is the difference between feature selection and feature extraction?

A: Feature Selection selects a SUBSET of existing features. Feature Extraction CREATES new features from existing ones (e.g., PCA, TF-IDF).

Q2. What is multicollinearity and why is it a problem?

A: When two or more features are highly correlated with each other. Causes instability in linear model coefficients. Detected via correlation matrix.

Q3. What does Mutual Information = 0 mean?

A: The feature and target are independent — the feature provides no information about the target. It should be removed.

Q4. What is the curse of dimensionality?

A: As the number of features increases, the data becomes sparse, distance metrics lose meaning, and models require exponentially more data. Feature selection combats this.

Q5. Which feature selection method is model-agnostic?

A: Filter methods like Correlation and Mutual Information do not depend on any specific ML model.

12. Conclusion

Feature selection is vital for building efficient, accurate, and interpretable models. Correlation analysis is the fastest but only detects linear relationships. Mutual Information is more powerful and detects non-linear dependencies. RFE systematically eliminates weak features using model feedback. Feature Importance from tree models provides built-in, efficient selection. Always validate performance improvement after selection.

PRACTICAL 7: Handling Imbalanced Data

1. Aim / Objective

To handle class imbalance in datasets using Oversampling, Undersampling, and SMOTE; and to compare model performance before and after balancing.

2. Theory

► Simple Explanation

Imagine a fraud detection dataset: 9,980 normal transactions and only 20 fraudulent ones. A model trained on this data will learn to always predict "normal" and achieve 99.8% accuracy — but it never detects any fraud! This is the class imbalance problem.

► Technical Explanation

Class imbalance occurs when one class has significantly more samples than another. The minority class (e.g., fraud, disease) is often the one we care about most. Standard accuracy is misleading — we need Precision, Recall, F1-score, and AUC-ROC.

3. Flowchart

```
START
↓
Check class distribution: y.value_counts()
↓
Imbalance detected (ratio > 4:1)?
↓
Choose Strategy:
  ├── Oversampling → Increase minority class samples
  │   └── SMOTE: Synthetic samples between real points
  ├── Undersampling → Reduce majority class samples
  │   └── RandomUnderSampler
  └── Combined: SMOTEENN, SMOTETomek
↓
Apply balancing ONLY on TRAINING data
↓
Train model → Evaluate with F1/Recall/AUC
↓
Compare: Before vs After balancing
END
```

4. Methods / Techniques

► 4.1 Random Oversampling

Duplicate random samples from the minority class until classes are balanced.

Advantages: Simple; no information loss.

Disadvantages: Overfitting — model memorizes duplicated samples.

► 4.2 Random Undersampling

Remove random samples from the majority class.

Advantages: Faster training; reduces overfitting risk from majority class.

Disadvantages: Discards potentially useful data.

► 4.3 SMOTE (Synthetic Minority Over-sampling Technique)

Creates SYNTHETIC (artificial) new samples by interpolating between existing minority class samples.

Algorithm:

1. For each minority sample, find K nearest minority neighbors
2. Randomly pick one neighbor
3. Create new synthetic sample at random point between them

Formula: $X_{\text{new}} = X_i + \lambda \times (X_{\text{nn}} - X_i)$ where $\lambda \in [0, 1]$

Advantages: Less overfitting than simple oversampling; creates diverse samples.

Disadvantages: Can create noisy samples near class boundaries.

5. Python Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, f1_score
# pip install imbalanced-learn
from imblearn.over_sampling import SMOTE, RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler

df = pd.read_csv("your_dataset.csv")
X = df.drop("target", axis=1).select_dtypes(include=[np.number])
y = df["target"]
print("Class distribution:", y.value_counts().to_dict())

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

def evaluate(name, X_tr, y_tr):
    rf = RandomForestClassifier(n_estimators=100, random_state=42)
    rf.fit(X_tr, y_tr)
    y_pred = rf.predict(X_test)
    f1 = f1_score(y_test, y_pred, average="weighted")
    print(f"\n=== {name} ===")
    print(classification_report(y_test, y_pred))
    return f1

# Baseline (no balancing)
f1_raw = evaluate("Baseline", X_train, y_train)

# Random Oversampling
ros = RandomOverSampler(random_state=42)
X_ros, y_ros = ros.fit_resample(X_train, y_train)
```

```
f1_ros = evaluate("Random Oversampling", X_ros, y_ros)

# Random Undersampling
rus = RandomUnderSampler(random_state=42)
X_rus, y_rus = rus.fit_resample(X_train, y_train)
f1_rus = evaluate("Random Undersampling", X_rus, y_rus)

# SMOTE
smote = SMOTE(random_state=42, k_neighbors=5)
X_sm, y_sm = smote.fit_resample(X_train, y_train)
f1_sm = evaluate("SMOTE", X_sm, y_sm)

print("\nF1-Score Comparison:")
print(f" Baseline: {f1_raw:.4f} | OverSample: {f1_ros:.4f} | UnderSample: {f1_rus:.4f} | SMOTE: {f1_sm:.4f}")
```

6. Code Explanation

- stratify=y → ensures same class ratio in train and test sets
- fit_resample() → applies balancing only on training data (never test!)
- classification_report → shows Precision, Recall, F1 per class
- f1_score(average="weighted") → accounts for class imbalance in evaluation
- SMOTE k_neighbors=5 → uses 5 nearest minority class samples to generate synthetic points

⚠ NEVER apply SMOTE or any resampling on the test set. Apply ONLY on training data!

7. Output Explanation

Expected Results (fraud detection example):

- Baseline: Accuracy 99% but Minority Recall $\approx$ 0.05 (misses most fraud)
- After SMOTE: Minority Recall $\approx$ 0.80 (detects 80% of fraud)
- F1-Score improves significantly for the minority class

8. Comparison Table

Method	Data Change	Overfitting Risk	Info Loss	Best For
Oversampling	More samples	High	No	Small datasets
Undersampling	Less samples	Low	Yes	Large datasets
SMOTE	Synthetic samples	Medium	No	Most cases
Class Weights	No change	Low	No	Quick fix

11. Viva Questions & Answers

Q1. Why is accuracy misleading for imbalanced data?

A: A model predicting all samples as majority class gets high accuracy but fails completely for minority class. Use F1-score, Precision, Recall, and AUC-ROC instead.

Q2. What is Recall and why is it critical in fraud detection?

A: Recall = $TP / (TP + FN)$. It measures how many actual positives are correctly identified. In fraud detection, missing a fraud (FN) is far worse than a false alarm.

Q3. How does SMOTE generate synthetic samples?

A: It interpolates between a minority sample and one of its K nearest minority neighbors. The new point lies on the line segment between the two samples.

Q4. What is the difference between class_weight and SMOTE?

A: class_weight penalizes the model more for misclassifying minority samples (no data change). SMOTE creates new synthetic data points to balance classes.

Q5. What is AUC-ROC?

A: Area Under the ROC Curve. Measures model ability to distinguish between classes. $AUC = 0.5 \rightarrow$ random; $AUC = 1.0 \rightarrow$ perfect. Better than accuracy for imbalanced data.

12. Conclusion

Class imbalance is a fundamental challenge in real-world ML applications like fraud detection, disease diagnosis, and spam filtering. SMOTE is the most popular and effective technique as it creates diverse synthetic samples without duplicating existing ones. Always evaluate imbalanced datasets using F1-score and AUC-ROC rather than accuracy. Apply resampling ONLY on training data.

 *Quick Fix: Use `class_weight="balanced"` in sklearn estimators when you cannot resample.*

PRACTICAL 8: Creating New Features (Feature Engineering)

1. Aim / Objective

To create new informative features using Feature Interaction, Polynomial Features, Date-Time Features, and Domain Knowledge-Based Features to improve model performance.

2. Theory

► Simple Explanation

Feature Engineering is the art of creating smarter input features. Instead of just giving a model "height" and "weight", we can give it $BMI = \text{weight} / \text{height}^2$ — a much more informative feature! Good feature engineering often matters more than choosing the right algorithm.

3. Flowchart

```
START
↓
Understand the dataset and domain
↓
Apply Feature Engineering strategies:
  ├── Interaction Features:    A × B, A / B, A - B
  ├── Polynomial Features:    A², A³, A×B
  ├── DateTime Features:      hour, day_of_week, is_weekend
  └── Domain Knowledge:        BMI, debt_ratio, age_group
↓
Add new columns to DataFrame
↓
Run Feature Selection to keep best new features
↓
Train model → Compare with original features
END
```

4. Methods / Techniques

► 4.1 Feature Interaction

Combining two features to capture their relationship.

- Product: $\text{price_per_sqft} = \text{price} / \text{area}$ (captures value density)
- Difference: $\text{age_of_loan} = \text{current_year} - \text{loan_start_year}$
- Ratio: $\text{debt_to_income} = \text{debt} / \text{income}$

These interactions often reveal non-linear relationships that simple features miss.

► 4.2 Polynomial Features

Creating higher-order features to capture non-linear patterns.

Example: From $[x_1, x_2] \rightarrow [1, x_1, x_2, x_1^2, x_1 \times x_2, x_2^2]$

degree=2 doubles the feature set; degree=3 may overfit.

PolynomialFeatures(degree=2, include_bias=False) — sklearn implementation.

► 4.3 Date-Time Features

Extracting useful components from timestamp columns.

- Hour, Day, Month, Year, Quarter
- Day of week (0=Monday), is_weekend
- Days since event, time since last purchase
- Is_holiday, season (Spring/Summer/Autumn/Winter)

► 4.4 Domain Knowledge Features

Features derived from expert knowledge about the problem domain.

- Healthcare: BMI = weight / height² (instead of raw weight & height)
- Finance: Debt-to-Income Ratio, Loan-to-Value Ratio
- E-commerce: Average order value, Return rate, Days since last purchase
- Text: Character count, Word count, Average word length

5. Python Code

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.read_csv("your_dataset.csv")

# — 1. Feature Interaction —
# Example: if dataset has "price" and "area" columns
if "price" in df.columns and "area" in df.columns:
    df["price_per_area"] = df["price"] / (df["area"] + 1e-5)

if "income" in df.columns and "debt" in df.columns:
    df["debt_ratio"] = df["debt"] / (df["income"] + 1e-5)
    df["income_minus_debt"] = df["income"] - df["debt"]

# — 2. Polynomial Features —
num_cols = df.select_dtypes(include=[np.number]).columns.drop("target",
errors="ignore")
poly = PolynomialFeatures(degree=2, include_bias=False, interaction_only=False)
X_poly = poly.fit_transform(df[num_cols])
poly_df = pd.DataFrame(X_poly, columns=poly.get_feature_names_out(num_cols))
print(f"Original features: {len(num_cols)} | After Poly: {poly_df.shape[1]}")

# — 3. Date-Time Features —
if "date" in df.columns:
    df["date"] = pd.to_datetime(df["date"])
    df["hour"] = df["date"].dt.hour
    df["day_of_week"] = df["date"].dt.dayofweek
    df["month"] = df["date"].dt.month
```

```

df["year"]          = df["date"].dt.year
df["is_weekend"]    = df["day_of_week"].isin([5, 6]).astype(int)
df["quarter"]       = df["date"].dt.quarter
df = df.drop("date", axis=1)

# — 4. Domain Knowledge —
if "weight" in df.columns and "height" in df.columns:
    df["BMI"] = df["weight"] / ((df["height"] / 100) ** 2 + 1e-5)
    df["BMI_category"] = pd.cut(df["BMI"], bins=[0,18.5,25,30,60],
                                labels=["Underweight","Normal","Overweight","Obese"])

# — Compare model before and after engineering —
X = df.drop("target", axis=1).select_dtypes(include=[np.number])
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
print(f"Accuracy with engineered features: {accuracy_score(y_test,
rf.predict(X_test)):.4f}")

```

6. Code Explanation

- 1e-5 added to divisor → prevents division by zero error
- PolynomialFeatures(degree=2) → creates squared terms and cross-products of all features
- interaction_only=True → creates only cross-products (no squared terms)
- pd.to_datetime() → converts string dates to datetime objects enabling .dt accessor
- .isin([5,6]).astype(int) → Saturday=5, Sunday=6 mapped to 1; other days to 0
- pd.cut() → bins continuous BMI into categorical ranges for better interpretability

7. Output Explanation

- Without engineering: Model accuracy 78%
- With interactions + datetime features: Accuracy 84%
- Feature importance shows new features (BMI, debt_ratio) rank highly
- Polynomial features increase columns from 10 to 65 (degree=2)

8. Feature Engineering Strategies

Strategy	Example	When to Use	Risk
Interaction	price/area, debt/income	Domain insight available	Low
Polynomial	x^2 , $x \times y$	Non-linear patterns	Overfitting
DateTime	is_weekend, hour	Time-series, logs	Low
Domain	BMI, CTR rate	Expert knowledge exists	Low

11. Viva Questions & Answers

Q1. What is feature engineering?

A: The process of creating new features from existing data using domain knowledge, mathematical transformations, and data manipulation to improve model performance.

Q2. Can polynomial features cause overfitting?

A: Yes. High-degree polynomials (degree 3+) create too many features, increasing model complexity and causing overfitting. Always use cross-validation and feature selection after polynomial expansion.

Q3. Why is datetime decomposition useful?

A: Raw timestamps are not directly useful. But extracted components (hour, day_of_week, is_weekend) capture cyclical patterns — e.g., sales spike on weekends, traffic peaks at rush hour.

Q4. What is the difference between feature extraction and feature engineering?

A: Feature extraction derives features from raw unstructured data (text → TF-IDF, image → CNN). Feature engineering creates new structured features from existing tabular data.

Q5. Why add 1e-5 to denominators?

A: To prevent division by zero. If area or height is 0, dividing would cause infinity or NaN. Adding a tiny constant ($1e-5 = 0.00001$) makes the calculation safe.

12. Conclusion

Feature engineering is one of the most impactful steps in ML — often more important than model selection. Interaction features capture relationships between variables. Polynomial features model non-linearity. DateTime decomposition reveals temporal patterns. Domain knowledge features encode expert understanding into the model. Always validate new features using Feature Selection and cross-validation.

 *Pro Tip: In Kaggle competitions, the winning edge often comes from creative feature engineering, not complex models!*

FINAL SUMMARY & BEST PRACTICES

Master Comparison: All Preprocessing Techniques

Practical	Technique	When to Use	Best Method	Avoid When
P1: Missing	Mean/Median/KNN	Data has NaN	KNNImputer	Very large data
P2: Normalize	MinMax/Standard	Distance/gradient algo	StandardScaler	Tree models
P3: Scale	Log/Power/Robust	Skewed features	PowerTransform	Normal data
P4: Encode	OHE/Label/Ordinal	Categorical data	OHE (nominal)	High cardinality OHE
P5: Extract	TF-IDF/W2V/CNN	Text or Image data	Word2Vec/CNN	Small text data
P6: Select	Corr/MI/RFE/RF	Many features	MI + RF Importance	Very few features
P7: Imbalance	SMOTE/Over/Under	Skewed class ratio	SMOTE	On test data
P8: Engineer	Interaction/Poly/DT	Domain knowledge	Domain-based	No domain context

Preprocessing Pipeline Best Practices

► Recommended Order of Operations

4. Remove duplicates → `df.drop_duplicates()`
5. Handle missing values → Imputer (fit on train only)
6. Encode categorical variables → OHE / Label / Ordinal
7. Handle class imbalance → SMOTE on training data only
8. Feature engineering → Create new informative features
9. Feature scaling → StandardScaler / MinMaxScaler (fit on train only)
10. Feature selection → Remove redundant/irrelevant features
11. Model training & evaluation

💡 Use `sklearn Pipeline` to chain all steps. This prevents data leakage and makes deployment easier.

Common Student Mistakes

► Critical Errors to Avoid:

- ⚠ Mistake 1: Fitting scaler/imputer on entire dataset (including test set) → DATA LEAKAGE
- ⚠ Mistake 2: Applying SMOTE on test data → inflates performance metrics artificially
- ⚠ Mistake 3: Using Label Encoding for nominal data with linear models → false ordinal relationship
- ⚠ Mistake 4: Dropping all rows with missing values → significant data loss, possible bias
- ⚠ Mistake 5: Forgetting to handle missing values before encoding → encoder crashes on NaN
- ⚠ Mistake 6: Using accuracy as primary metric for imbalanced data → misleading results
- ⚠ Mistake 7: Creating polynomial features without feature selection after → curse of dimensionality
- ⚠ Mistake 8: Applying log transform to columns with negative/zero values → NaN/error

Interview-Focused Key Points

► Must-Know Concepts for Interviews & Exams:

- Data Leakage: Fitting preprocessing steps on test data. Always fit on train, transform test.
- Stratified Split: Use `stratify=y` in `train_test_split` for imbalanced datasets.
- Pipeline: `sklearn.pipeline.Pipeline` chains all preprocessing + model steps cleanly.
- Cross-validation: Always use K-Fold CV for reliable performance estimation.
- Metrics for Imbalance: Use F1-score, Precision, Recall, AUC-ROC — not accuracy alone.
- Dummy Variable Trap: Use `drop_first=True` in `pd.get_dummies()` to avoid multicollinearity.
- IQR: Interquartile Range = $Q3 - Q1$, used in `RobustScaler` and outlier detection.
- Gini Impurity: Used by Random Forest to calculate feature importance.
- Cardinality: Number of unique values in a categorical column.
- Variance Inflation Factor (VIF): Detects multicollinearity numerically ($VIF > 5$ = problem).

Exam Important Points

► Short Notes for Quick Revision:

- KNNImputer: K-Nearest Neighbors based imputation. `n_neighbors=5` default. Slow but accurate.
- SMOTE: Synthetic Minority Over-sampling Technique. Creates synthetic points, not duplicates.
- TF-IDF: Term Frequency $\times$ Inverse Document Frequency. Sparse vector. Bag-of-words based.
- Word2Vec: Neural network trained to predict context words. Dense semantic vectors.
- One-Hot: $N-1$ columns for N categories. Avoids dummy variable trap with `drop_first=True`.
- StandardScaler: `mean=0`, `std=1`. Not bounded. Best for SVM, PCA, Logistic Regression.
- MinMaxScaler: Maps to $[0, 1]$. Best for Neural Networks. Sensitive to outliers.
- RobustScaler: Uses median & IQR. Best for data with outliers.
- RFE: Recursive Feature Elimination. Wrapper method. Uses estimator internally.
- Mutual Information: Measures dependency. Works for both linear and non-linear relations.

END OF MACHINE LEARNING PRACTICAL MANUAL

Data Preprocessing & Feature Engineering | All 8 Practicals Covered